Visual Studio and in the CLI by configuring an xunit.runner.json file, as described at https://xunit.net/docs/configuration-files.

Instead of creating a `[Fact]` test method, you can create a `[Theory]` test method. A theory provides a way of parameterizing your test methods, effectively taking your test method and running it multiple times with different arguments. Each set of arguments is considered a different test.

You could rewrite the `[Fact]` test in listing 35.4 to be a `[Theory]` test, as shown in the next listing. Instead of specifying the variables in the method body, pass them as parameters to the method and then decorate the method with three `[InlineData]` attributes. Each instance of the attribute provides the parameters for a single run of the test.

Listing 35.5 Theory test for `ConvertToGbp` testing multiple sets of values

```
[Theory]                                                    ❶
[InlineData(0, 3, 0)]                                       ❷
[InlineData(3, 1.5, 2)]                                     ❷
[InlineData(3.75, 2.5, 1.5)]                                ❷
public void ConvertToGbp_ConvertsCorrectly (                ❸
    decimal value, decimal rate, decimal expected)          ❸
{
    var converter = new CurrencyConverter();
    int dps = 4;                                            ❹

    var actual = converter.ConvertToGbp(value, rate, dps); ❺

    Assert.Equal(expected, actual);                         ❻
}
```

❶ Marks the method as a parameterized test

❷ Each [InlineData] attribute provides all the parameters for a single run of the test method.

❸ The method takes parameters, which are provided by the [InlineData] attributes.

❹ The dps variable doesn’t change, so there’s no need to include it in [InlineData].

❺ Executes the SUT

❻ Verifies the result